

RAJALAKSHMI ENGINEERING COLLEGE
AN AUTONOMOUS INSTITUTION
Affiliated to ANNA UNIVERSITY
Rajalakshmi Nagar, Thandalam,
Chennai-602105



DEPARTMENT OF COMPUTER SCIENCE
AND ENGINEERING

CS23A34 - USER INTERFACE DESIGN LABORATORY
ACADEMIC YEAR:2024-2025 (EVEN)

INDEX

Reg. No : 230701292

Name : SANJEEV ADITYA.P

Branch : CSE

Year/Section : II-D

LIST OF EXPERIMENTS

Experiment No:	Title	Tools
1	Design a UI where users recall visual elements (e.g., icons or text chunks). Evaluate the effect of chunking on user memory.	Figma.
2.	Develop and compare CLI, GUI, and Voice User Interfaces (VUI) for the same task and assess user satisfaction.	Python (Tkinter for GUI, Speech Recognition for VUI) / Terminal
3	A) Create a prototype with familiar and unfamiliar navigation elements. Evaluate ease of use with different user groups.	Proto.io
	B) Create a prototype with familiar and unfamiliar navigation elements. Evaluate ease of use with different user groups.	Wireflow
4	A) Conduct task analysis for an app (e.g., online shopping) and document user flows. Create corresponding wireframes.	Lucid chart (free tier)
	B) Conduct task analysis for an app (e.g., online shopping) and document user flows. Create corresponding wireframes.	Dia (open source).
5.	A) Simulate the lifecycle stages for UI design using the RAD model and develop a small interactive interface.	Axure RP
	B) Simulate the lifecycle stages for UI design using the RAD model and develop a small interactive interface.	OpenProj.

6.	Experiment with different layouts and color schemes for an app. Collect user feedback on aesthetics and usability.	GIMP (open source for graphics).
7.	A)Develop low-fidelity paper prototypes for a banking app and convert them into digital wireframes.	Pencil Project
	B)Develop low-fidelity paper prototypes for a banking app and convert them into digital wireframes.	Inkscape.
8.	A) Create storyboards to represent the user flow for a mobile app (e.g., food delivery app).	Balsamiq
	B) Create storyboards to represent the user flow for a mobile app (e.g., food delivery app).	OpenBoard
9.	Design input forms that validate data (e.g., email, phone number) and display error messages.	HTML/CSS, JavaScript (with Validator.js).
10.	Create a data visualization (e.g., pie charts, bar graphs) for an inventory management system.	Java Script

Experiment 1

Good and Bad Design

AIM:

Create a prototype with familiar and unfamiliar navigation elements. Evaluate ease of use with different user groups.

MATERIALS REQUIRED:

Pencil Project, Figma.

PROCEDURE:

Step 1: Sign Up and Create a New Project

1. Go to figma.com and create an account (if you haven't already).
2. Once logged in, click ;New File; to start a blank project.
3. You'll see a blank canvas where you can start designing.

Step 2: Create the Frame (Artboard)

1. On the left toolbar, select the Frametool (shortcut: F).
2. Choose a mobile preset (e.g., iPhone 13) from the right-hand panel.
3. A mobile-sized frame will appear on the canvas, which will act as your app screen.

Step 3: Design the Login Screen

Add a Background Color:

1. Select the frame and go to the right-side panel.
2. Under “Fill” choose a background color (e.g., light blue #E3F2FD).

Insert a Logo:

1. Click the “Rectangle” tool (shortcut: R) and draw a placeholder for a logo.
2. Use the “Text” tool (shortcut: T) to add your app name, e.g., MyApp.
3. Adjust font size and color from the right-hand panel.

Add Input Fields:

1. Use the Rectangle tool to draw two boxes for username and password fields.
2. Add placeholder text inside (e.g., “Enter your email”).
3. Apply rounded corners under Corner Radius in the right panel.

Add a Login Button:

1. Create a button using the Rectangle tool and set the color to blue (#1E88E5).
2. Use the Text tool to add the text Login inside the button.
3. Group the button and text together by selecting them and pressing Ctrl + G (Windows) or Cmd + G (Mac).

Align Elements:

Use the alignment tools in the top menu (center everything vertically and horizontally).

Adjust spacing between elements using the Auto Layout feature (Shift + A).

Step 4: Prototyping the Interaction

1. Click the Prototype tab on the right panel.
2. Select the "Login" button and drag the blue dot to a new frame (e.g., a home screen).
3. Set the interaction to On Click → Navigate to the next screen.
4. Choose an animation effect (e.g., Smart Animate).

Step 5: Preview the Design

1. Click the Play button in the top-right corner to preview your app prototype.
2. Try clicking on the login button to see the transition to the next screen.

Step 6: Share Your Design

1. Click the Share button in the top-right corner.
2. You can invite team members via email or generate a shareable link.
3. Adjust permissions (View, Edit, or Comment only).

Step 7: Export Assets

1. Select the elements you want to export (e.g., the logo or button).
2. In the right-hand panel, click Export and choose a format (PNG, JPG, SVG).
3. Click "Export" to download assets for developers.

OUTPUT:

BAD DESIGN

iPhone 13 & 14 - 1



» Login



Password



User name



GOOD DESIGN:



Welcome back
sign in to access your account

Enter your email 

Password 

☐ Remember me [Forget password ?](#)

Next >

New Member? [Register now](#)

EXPLANATION:

In Bad design there is no proper background chosen and attributes such as user name,password and login whereas in the Good design we implement good background with Username ,password,phone number and login id. No detailed explanation provided.Mismatched attributes and icons are not proper.

RESULT:

The design of the prototype with familiar and unfamiliar navigation elements and Evaluating the ease of use with different user groups is successfully designed.

Experiment 2

Chunking

AIM:

To design a UI where users recall visual elements (e.g., icons or text chunks). Evaluate the effect of chunking on user memory.

PROCEDURE:

1. **Create Home Screen:**
 - Add a **1024x768px frame** (File → New Frame).
 - Insert a **title ("Memory Recall Task")** and instructions using the **Text Tool (T)**.
 - Design a **"Start" button** (Rectangle + Text) and link it to the Chunking Phase via **Prototype mode**.
2. **Set Up Chunking Phase:**
 - Create a new **frame** for the chunking display.
 - Add **icons or text** that users need to remember.
3. **Apply Chunking Techniques:**
 - **Chunking with Borders:** Group 3-5 items using **Rectangles (R)**.
 - **Chunking without Borders:** Place items close together without clear separation.
4. **Simulate Viewing Time:**
 - Select the **Chunking Phase frame**, go to **Prototype mode**, and set an **"After Delay" transition (5000ms)** to the Recall Phase.
5. **Create Recall Phase UI:**
 - Add a new **frame** for user input.
 - Add a question: **"Select the items you remember seeing."**
6. **Design Recall Options:**
 - **Multiple-choice method:** Add checkboxes/radio buttons.
 - **Text input method:** Create labeled text input fields (e.g., "Item 1").
7. **Create Submit Button:**
 - Design a **"Submit Recall" button** (Rectangle + Text).
 - Link it to the Result Screen in **Prototype mode**.
8. **Create Result Screen:**
 - Add a **title** (e.g., "Your Recall Score") and feedback text (e.g., "You recalled 4/5 items!").

9. **Provide Analysis:**

- Test different chunk sizes (3 vs. 5 items) and content types (icons vs. text).

10. **Final Testing & Sharing:**

- Click **Play** to preview the prototype.
- Use the **Share** button to invite testers.

OUTPUT:-

PAGE:-1 START PAGE





Remember the given items Below



⌚ 00:00:05

**Select the items you
remember seeing**



Submit

EXPERIMENT 03: CLI,GUI AND VUI

AIM: The aim is to develop and compare Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) for the same task, and assess user satisfaction using Python (with Tkinter for GUI and Speech Recognition for VUI) and Terminal.

PROCEDURE:

1. Implement a **CLI To-Do List** in Python for adding, viewing, and removing tasks via terminal input.
2. Develop a **GUI To-Do List** using Tkinter with text input, buttons, and a task list display.
3. Build a **VUI To-Do List** using SpeechRecognition and pyttsx3 for voice commands.
4. Compare user experience across CLI, GUI, and VUI interfaces.

PROGRAM:

CLI- Command line interface

```
tasks=[]
def add_task(task):
    tasks.append(task)
    print(f"Task '{task}' added.")

def view_tasks():
    if tasks:
        print("Your tasks:")
        for idx,task in enumerate(tasks,1):
            print(f"{idx}.{task}")

    else:
        print("No tasks to show.")
def remove_task(task_number):
    if 0< task_number <= len(tasks):
        removed_task=tasks.pop(task_number-1)
        print(f"Task '{removed_task}' removed.")
    else:
        print("Invalid task number.")
def main():
    while True:
        print("\nOptions: 1. Add Task 2.View Tasks 3.Remove Task 4.Exit")
        choice=input("enter your choice:")

        if choice=='1.':
            task=input("Enter task: ")
```

```

        add_task(task)
    elif choice=='2.':
        view_tasks()
    elif choice == '3.':
        task_number=int(input("Enter task number to remove: "))
        remove_task(task_number)
    elif choice == '4.' :
        print("Exiting..")
        break
    else:
        print("Invalid choice. Please try again.")
if __name__=="__main__":
    main()

```

```

AMD64]] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/HDC0422041/Documents/CLI 312.py =====

Options: 1. Add Task 2.View Tasks 3.Remove Task 4.Exit
enter your choice:2.
No tasks to show.

Options: 1. Add Task 2.View Tasks 3.Remove Task 4.Exit
enter your choice:1.
Enter task: we
Task 'we'added.

Options: 1. Add Task 2.View Tasks 3.Remove Task 4.Exit
enter your choice:1.
Enter task: the
Task 'the'added.

Options: 1. Add Task 2.View Tasks 3.Remove Task 4.Exit
enter your choice:1.
Enter task: mee
Task 'mee'added.

Options: 1. Add Task 2.View Tasks 3.Remove Task 4.Exit
enter your choice:2.
Your tasks:
1.we
2.the
3.mee

Options: 1. Add Task 2.View Tasks 3.Remove Task 4.Exit
enter your choice:3.
Enter task number to remove: 1
Task 'we'removed.

Options: 1. Add Task 2.View Tasks 3.Remove Task 4.Exit
enter your choice:4.
Exiting..
>>>

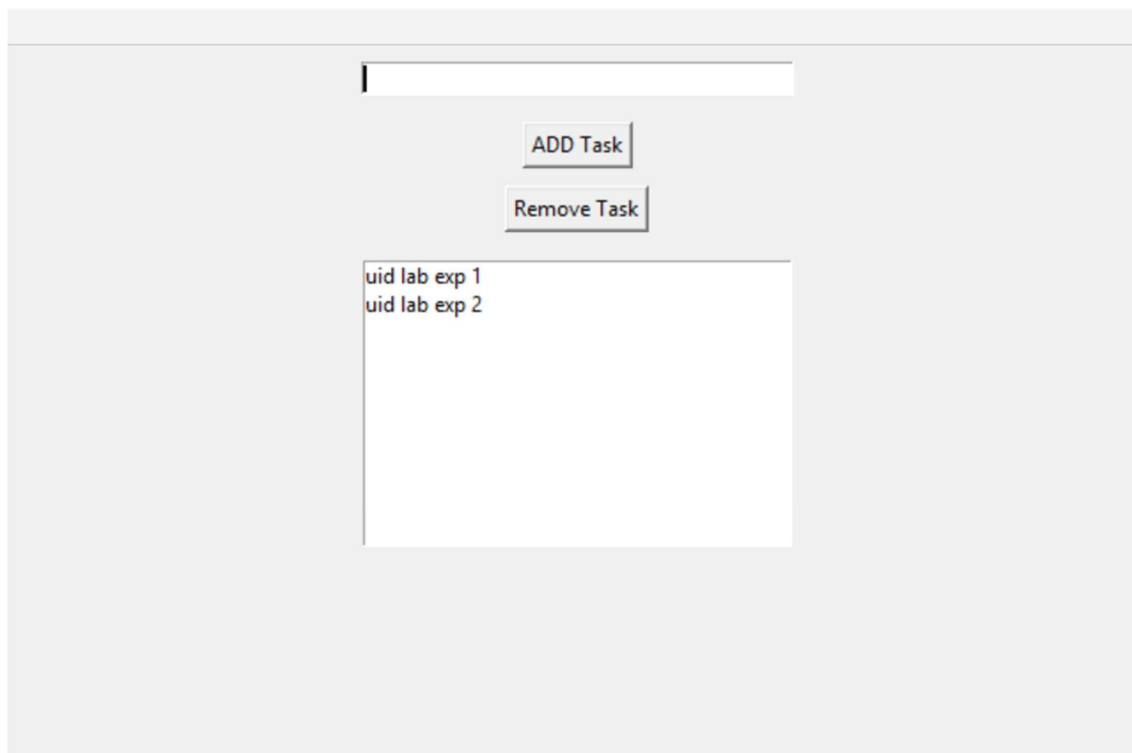
```

GUI – Graphical User Interface

```
import tkinter as tk
```

```
from tkinter import messagebox
tasks = []
def add_task():
    task = task_entry.get()
    if task:
        tasks.append(task)
        task_entry.delete(0, tk.END)
        update_task_list()
    else:
        messagebox.showwarning("Warning", "Task cannot be empty.")
def update_task_list():
    task_list.delete(0, tk.END)
    for task in tasks:
        task_list.insert(tk.END, task)
def remove_task():
    selected_task_index = task_list.curselection()
    if selected_task_index:
        task_list.delete(selected_task_index)
        tasks.pop(selected_task_index[0])
app = tk.Tk()
app.title("To-Do List")
task_entry = tk.Entry(app, width=40)
task_entry.pack(pady=10)
add_button = tk.Button(app, text="ADD Task", command=add_task)
add_button.pack(pady=5)
remove_button = tk.Button(app, text="Remove Task", command=remove_task)
remove_button.pack(pady=5)
task_list = tk.Listbox(app, width=40, height=10)

task_list.pack(pady=10)
app.mainloop()
```



VUI – Voice User Interface

```
import speech_recognition as sr
import pyttsx3

tasks = []
recognizer = sr.Recognizer()
engine = pyttsx3.init()

def add_task(task):
    tasks.append(task)
    engine.say(f"Task {task} added")
    engine.runAndWait()

def view_tasks():
    if tasks:
        engine.say("Your tasks are")
        for task in tasks:
            engine.say(task)
    else:
        engine.say("No tasks to show")
    engine.runAndWait()

def remove_task(task_number):
    if 0 < task_number <= len(tasks):
        removed_task = tasks.pop(task_number - 1)
```

```

        engine.say(f"Task {removed_task} removed")
    else:
        engine.say("Invalid task number")
    engine.runAndWait()

def recognize_speech():
    with sr.Microphone() as source:
        print("Listening...")
        audio = recognizer.listen(source)
        try:
            command = recognizer.recognize_google(audio)
            return command
        except sr.UnknownValueError:
            engine.say(";Sorry, I did not understand that")
            engine.runAndWait()
            return None

def main():
    while True:
        engine.say("Options: add task, view tasks, remove task, or exit")
        engine.runAndWait()
        command = recognize_speech()
        if not command:
            continue
        if "add task" in command:
            engine.say("What is the task?")
            engine.runAndWait()
            task = recognize_speech()
            if task:
                add_task(task)
        elif "view tasks" in command:
            view_tasks()
        elif "remove task" in command:
            engine.say(";Which task number to remove?")
            engine.runAndWait()
            task_number = recognize_speech()
            if task_number:
                remove_task(int(task_number))
        elif "exit" in command:
            engine.say(";Exiting...")
            engine.runAndWait()
            break
        else:
            engine.say(";Invalid option. Please try again.")
            engine.runAndWait()

if __name__ == "__main__":
    main()

```

```
Listening...
```

```
Listening...
```

```
Listening...
```

```
Listening...
```

```
Listening...
```

```
Listening...
```

```
Listening...
```

```
Listening...
```

```
Listening...
```

```
Listening...
```

RESULT:

Thus the to do list for GUI CLI,VUI has been implemented successfully .

Excercise 3b

Date:

Create a prototype with familiar and unfamiliar navigation elements. Evaluate ease of use with different user groups using wireflow

AIM:

The aim is to design a prototype with both well-known and new navigation elements and measure user-friendliness across different user groups using Wireflow.

PROCEDURE:

Tool link: <https://wireflow.co/>

Step 1: Plan Your Prototype

1. Define Navigation Elements:
 - *Familiar*: Standard menus, top bars, footers, and sidebar navigation.
 - *Unfamiliar*: Novel features such as hidden menus, gesture-based navigation, or custom swipes.
2. Sketch Your Layout:
 - Start with paper sketches or use tools like Figma or Sketch to visualize your design concepts.

Step 2: Set Up Your Wireflow Project

1. Sign Up/Log In:
 - Head to Wireflow and create an account or log in if you already have one.
2. Start a New Project:
 - Click on "New Project" and name it. Choose a template or start from scratch.

Step 3: Design the Prototype

1. Add Familiar Navigation Elements:
 - Drag and drop components like menus, header bars, buttons, etc., into your screens.
2. Incorporate Unfamiliar Elements:
 - Introduce hidden menus, unique gestures, or unexpected interactions.
3. Link Screens:
 - Use Wireflow's linking tools to create connections and transitions between screens.

Step 4: Prepare for Usability Testing

1. Identify User Groups:
 - Segment users based on age, tech-savviness, or previous experience with similar products.
2. Recruit Participants:
 - Use online tools like UserTesting, forums, or social media to find participants.

Step 5: Conduct Testing

1. Share the Prototype:
 - Invite users to interact with your prototype via a shareable link from Wireflow.
2. Test Sessions:
 - Ask users to complete tasks using both types of navigation. Observe their interactions and collect feedback.
3. Collect Feedback:
 - Utilize Wireflow's feedback features or conduct follow-up interviews to gather detailed responses.

Step 6: Analyze and Report

1. Analyze Data:

- Review the feedback and data collected. Look for patterns in ease of use and user preferences.

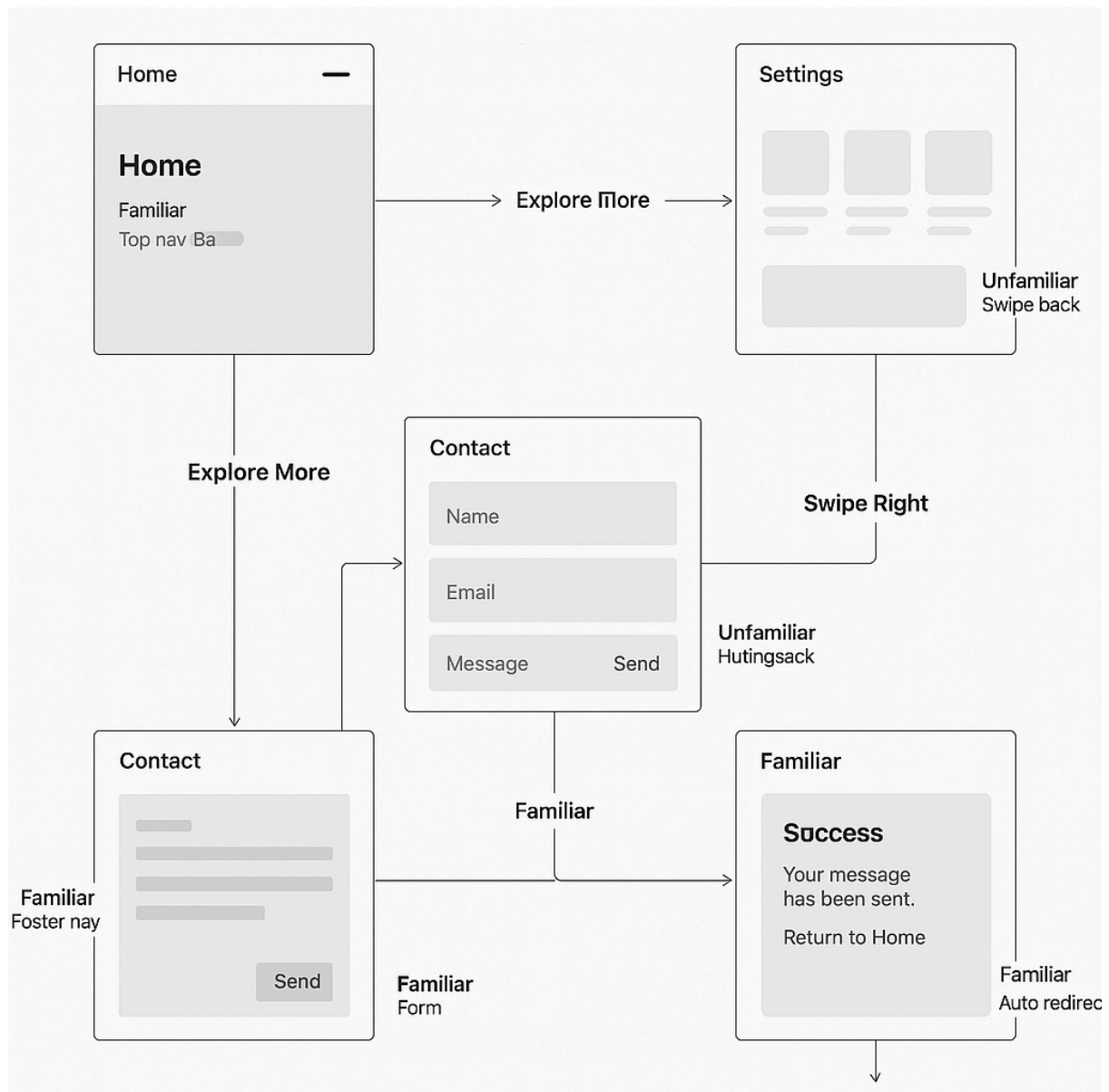
2. Compare Results:

- Compare how different user groups interacted with familiar vs. unfamiliar navigation.

3. Create a Report:

- Summarize your findings, highlighting insights, challenges, and recommendations

OUTPUT:-



RESULT:-

Hence the Familiar and unfamiliar using wireflow is implemented and executed.

EXERCISE 4

Create a prototype with familiar and unfamiliar navigation elements. Evaluate ease of use with different user groups using proto.io

AIM:

The aim is to develop a prototype incorporating both familiar and novel navigation elements and assess usability among diverse user groups using Proto.io.

PROCEDURE:

i) Example 1:

Tool Link: <https://proto.io/>

Step 1: Sign Up and Log In

1. Go to proto.io.
2. Sign up for a new account or log in if you already have one.

Step 2: Create a New Project

1. Click on "Create New Project."
2. Give your project a name (e.g., "Simple App Example").
3. Select the device type (e.g., Mobile - iPhone X).
4. Click "Create" to start the project.

Step 3: Design the Home Screen

1. Add a New Screen:
 - Click on the "+" button in the left panel to add a new screen.
 - Choose "Blank" and name it "Home."
2. Add Elements to the Home Screen:

- Drag a "Header" widget from the "Widgets" panel to the top of the screen.
 - Double-click the header to edit the text and change it to "Home Screen."
 - Drag a "Button" widget onto the screen. Place it in the center.
 - Double-click the button to edit the text and change it to "Go to Profile."
3. Add Interaction:
- Select the button and click on the "Interactions" tab on the right panel.
 - Click "+ Add Interaction."
 - Set the trigger to "Tap/Click."
 - Set the action to "Navigate to Screen" and choose "New Screen."
 - Create a new screen and name it "Profile."

Step 4: Design the Profile Screen

1. Add Elements to the Profile Screen:
- On the newly created Profile screen, drag a "Header" widget to the top of the screen.
 - Double-click the header to edit the text and change it to "Profile Screen."
 - Drag an "Image" widget onto the screen. Place it below the header.
 - Double-click the image to upload a profile picture or any placeholder image.
 - Drag a "Text" widget onto the screen to add some profile information (e.g., "John Doe, Software Engineer").
2. Add Back Button:
- Drag a "Button" widget onto the screen.
 - Double-click the button to edit the text and change it to "Back to Home."
3. Add Interaction:
- Select the button and click on the "Interactions" tab on the right panel.
 - Click "+ Add Interaction."
 - Set the trigger to "Tap/Click."

- Set the action to "Navigate to Screen" and choose "Home."

Step 5: Preview the Prototype

1. Click on the "Preview" button in the top-right corner.
2. Interact with the prototype by clicking on the buttons to navigate between the Home and Profile screens.

Step 6: Share the Prototype

1. Click on the "Share" button in the top-right corner.
2. Copy the shareable link and send it to others for feedback.

i) Example 2:

Step 1: Plan Your Prototype

1. Identify Your Elements:

- *Familiar*: Common navigation elements such as a top menu bar, side panels, breadcrumb trails, and footer links.
- *Unfamiliar*: Experiment with things like hidden menus, gesture-based navigation, or voice commands.

2. Sketch Out Your Concept:

- Draft wireframes on paper, using tools like Figma or Sketch to visualize how both elements will coexist.

Step 2: Start Your Project on Proto.io

1. Sign Up/Log In:

- Go to Proto.io and either create an account or log in if you already have one.

2. Create New Project:

- Click on the "Create a new project" button, select the type of project, and give it a name.

3. Choose a Template:

- Select a template that suits your needs or start from scratch.

Step 3: Design Your Screens

1. Familiar Navigation:

- Drag and drop elements like menus, tabs, buttons that users are accustomed to.

2. Unfamiliar Navigation:

- Add unique elements such as swipe gestures, hover interactions, or voice commands.

3. Link Screens:

- Use Proto.io's interaction design tools to set up transitions between screens.

Step 4: Gather User Groups

1. Define User Groups:

- Segment users into different categories such as age group, tech-savviness, or experience with similar products.

2. Recruit Participants:

- Use platforms like UserTesting, surveys, or social media to find participants.

Step 5: Conduct Usability Testing

1. Deploy the Prototype:

- Share the unique project link or invite users to test your prototype directly through Proto.io.

2. Test Sessions:

- Conduct usability tests with users from each group, giving them specific tasks to accomplish.

3. Collect Feedback:

- Use Proto.io's feedback tools or conduct interviews to gather their thoughts and experiences.

Step 6: Analyze and Evaluate

1. Data Analysis:

- Look at how users interacted with each element. Use Proto.io's analytics tools to draw insights.

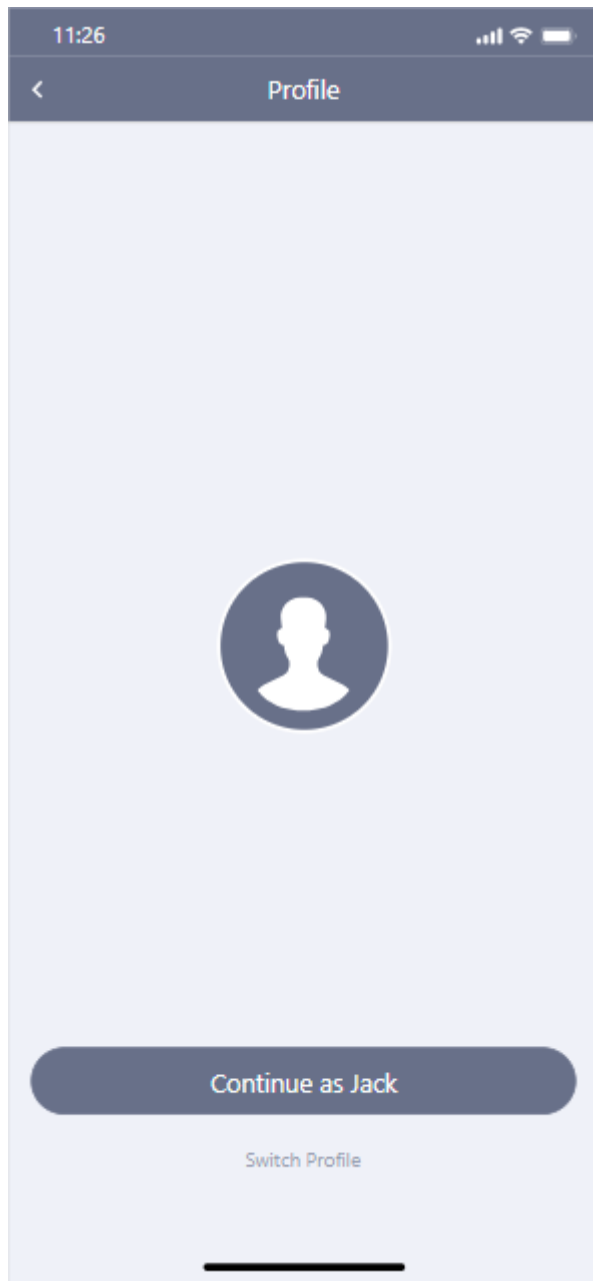
2. Compare Groups:

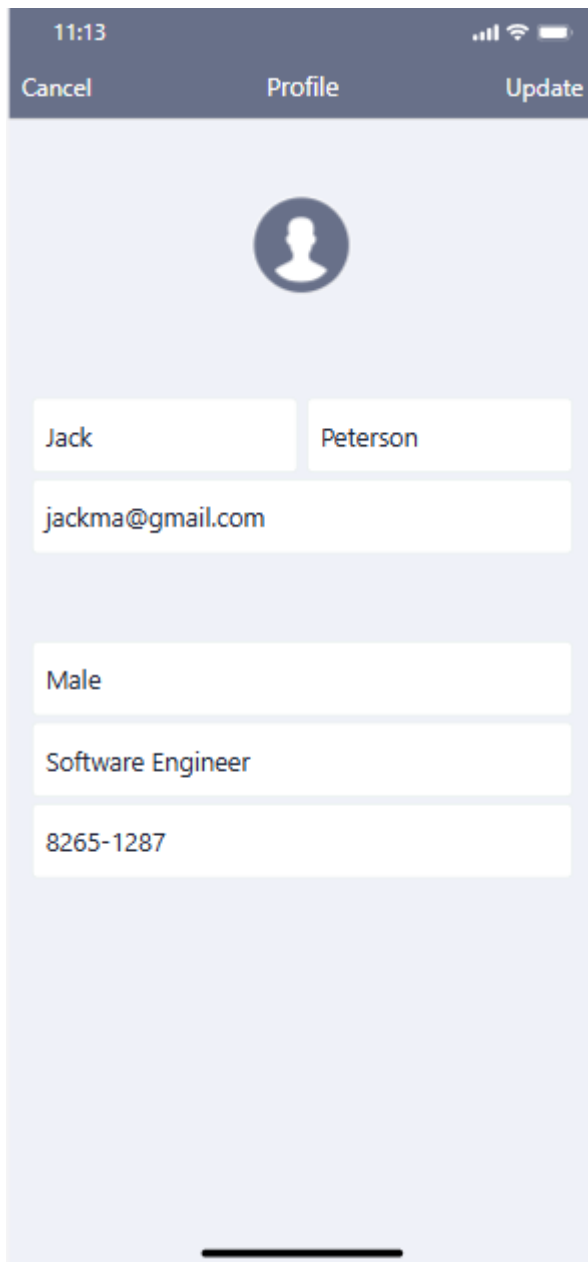
- Compare how different user groups responded to familiar vs. unfamiliar navigation.

3. Report Findings:

- Summarize the results in a detailed report highlighting key insights, pain points, and recommendations.

OUTPUT:





A mobile application prototype for a user profile. The interface features a dark blue header bar with the time '11:13' on the left and signal, Wi-Fi, and battery status icons on the right. Below the header, the word 'Profile' is centered, flanked by 'Cancel' on the left and 'Update' on the right. The main content area has a light blue background. At the top of this area is a circular profile picture placeholder with a white silhouette on a dark blue background. Below the profile picture are four white input fields. The first field is split into two parts: 'Jack' on the left and 'Peterson' on the right. The second field contains the email address 'jackma@gmail.com'. The third field contains the gender 'Male'. The fourth field contains the profession 'Software Engineer'. Below these fields is a fifth white input field containing the phone number '8265-1287'. A black horizontal line at the bottom of the screen represents the mobile home indicator bar.

11:13

Cancel Profile Update

Profile Picture Placeholder

Jack Peterson

jackma@gmail.com

Male

Software Engineer

8265-1287

RESULT:

Hence the prototype with familiar and unfamiliar navigation elements is designed and implemented.

Excercise 4b

Date:

Conduct task analysis for an app (e.g., online shopping) and document user flows. Create corresponding wireframes using dia

AIM:

The aim is to perform task analysis for an app, such as online shopping, document user flows, and create corresponding wireframes using Dia.

PROCEDURE:

Tool link: <http://dia-installer.de/>

1. Install Dia:

- Download Dia from the official website (<http://dia-installer.de/>)
- Install Dia on your computer
- Open Dia:
- Launch the Dia application.

2. Create New Diagram:

- Go to File -> New Diagram.
- Select Flowchart as the diagram type.

3. Add Shapes:

- Use the shape tools (rectangles, ellipses, etc.) to create wireframes for each screen.

■ For example:

- Home Page: Rectangle
- Product Categories: Rectangle
- Product Listings: Rectangle
- Product Details: Rectangle
- Cart: Rectangle

- Checkout: Rectangle
- Order Confirmation: Rectangle
- Order History: Rectangle

4. Connect Shapes:

- Use the line tool to connect shapes, representing the user flows.
 - For example:
 - Home Page -> Product Categories
 - Product Categories -> Product Listings
 - Product Listings -> Product Details
 - Product Details -> Cart
 - Cart -> Checkout
 - Checkout -> Order Confirmation
 - Order Confirmation -> Order History

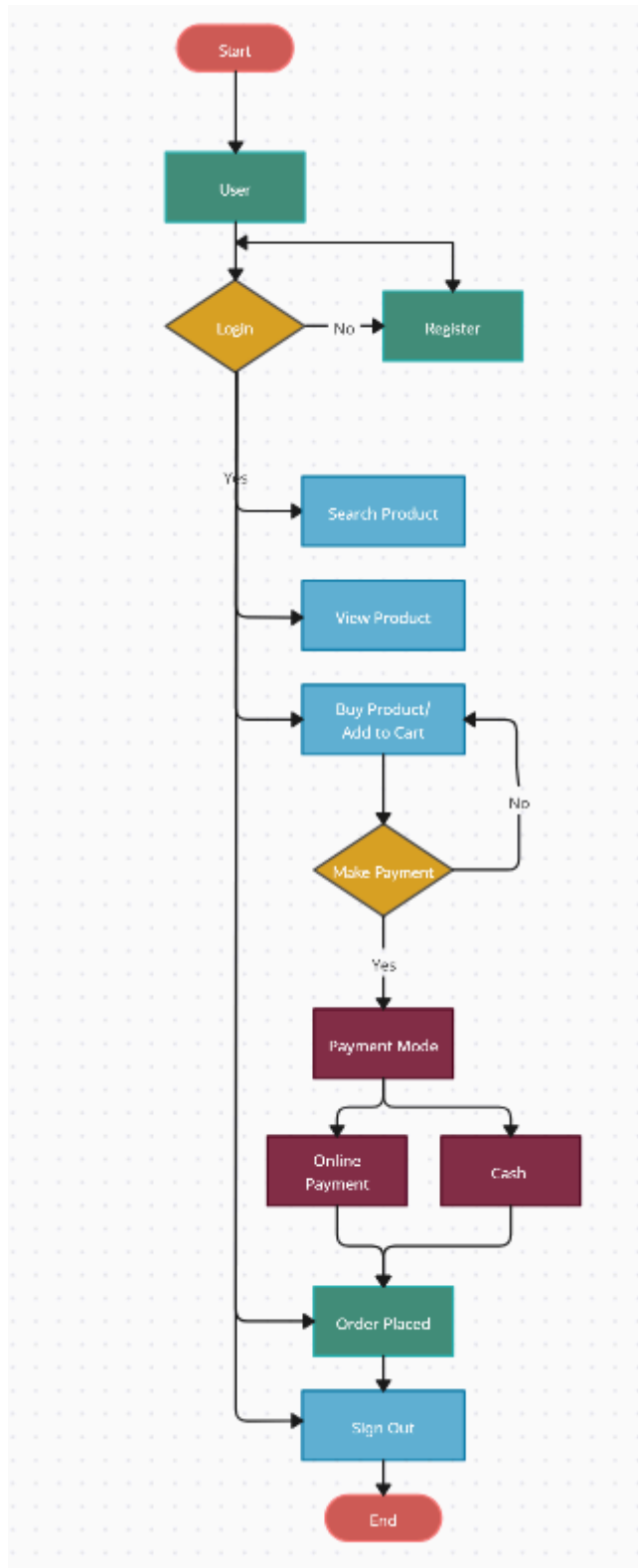
5. Label Shapes:

- Double-click on each shape to add labels.
 - For example:
 - Label the rectangle as "Home Page", "Categories", "Product Listings", "Product Details", "Cart", "Checkout", "Order Confirmation", "Order History".

6. Save the Diagram:

- Go to File -> Save As.
- Save the diagram with a meaningful name, such as "Online Shopping App User Flows".

OUTPUT:



RESULT:

Hence the Flow chart for online shopping is implemented using dia application.

Exercise 5A

Date:

Simulate the lifecycle stages for UI design using the RAD model and develop a small interactive interface using Axure RP

AIM:

The aim is to demonstrate the lifecycle stages of UI design via the RAD model and develop a small interactive interface employing Axure RP.

PROCEDURE:

Tool Link: <https://www.axure.com/>

Simulating the Lifecycle Stages for UI Design Using the RAD Model

RAD Model (Rapid Application Development): The RAD model emphasizes quick development and iteration. It consists of the following phases:

1. Requirements Planning:
 - Gather initial requirements and identify key features of the UI.
 - Engage stakeholders to understand their needs and expectations.
2. User Design:
 - Create initial prototypes and wireframes.
 - Conduct user feedback sessions to refine the designs.
 - Use tools like Axure RP to develop interactive prototypes.
3. Construction:
 - Develop the actual UI based on the refined designs.
 - Perform iterative testing and feedback cycles.
4. Cutover:
 - Deploy the final UI.

- Conduct user training and support.

Axure RP Interactive Interface Development

Phase 1: Requirements Planning

1. Identify Key Features:

- Navigation (Home, Product Categories, Product Details, Cart, Checkout, Order Confirmation, Order History)
- User actions (Browsing, Searching, Adding to Cart, Checkout, Tracking Orders)

2. Create a Requirements Document:

- List all features and functionalities.
- Document user stories and use cases.

Phase 2: User Design

1. Install and Launch Axure RP:

- Download and install Axure RP from Axure's official website.
- Launch the application.

2. Create a New Project:

- Go to File -> New to create a new project.
- Name the project (e.g., "Shopping App Interface").

3. Create Wireframes:

- Use the widget library to drag and drop elements onto the canvas.
- Design wireframes for each screen:
 - Home Page
 - Product Categories
 - Product Listings
 - Product Details
 - Cart
 - Checkout

- Order Confirmation
- Order History

4. Add Interactions:

- Select an element (e.g., button) and go to the Properties panel.
- Click on Interactions and choose an interaction (e.g., OnClick).
- Define the action (e.g., navigate to another screen).

5. Create Masters:

- Create reusable components (e.g., headers, footers) using Masters.
- Drag and drop masters onto the wireframes.

6. Add Annotations:

- Add notes to describe each element's purpose and functionality.
- Use the Notes panel to add detailed annotations.

Phase 3: Construction

1. Develop Interactive Prototypes:

- Convert wireframes into interactive prototypes by adding interactions and transitions.
- Use dynamic panels to create interactive elements (e.g., carousels, pop-ups).

2. Test and Iterate:

- Preview the prototype using the Preview button.
- Gather feedback from users and stakeholders.
- Make necessary adjustments based on feedback.

Phase 4: Cutover

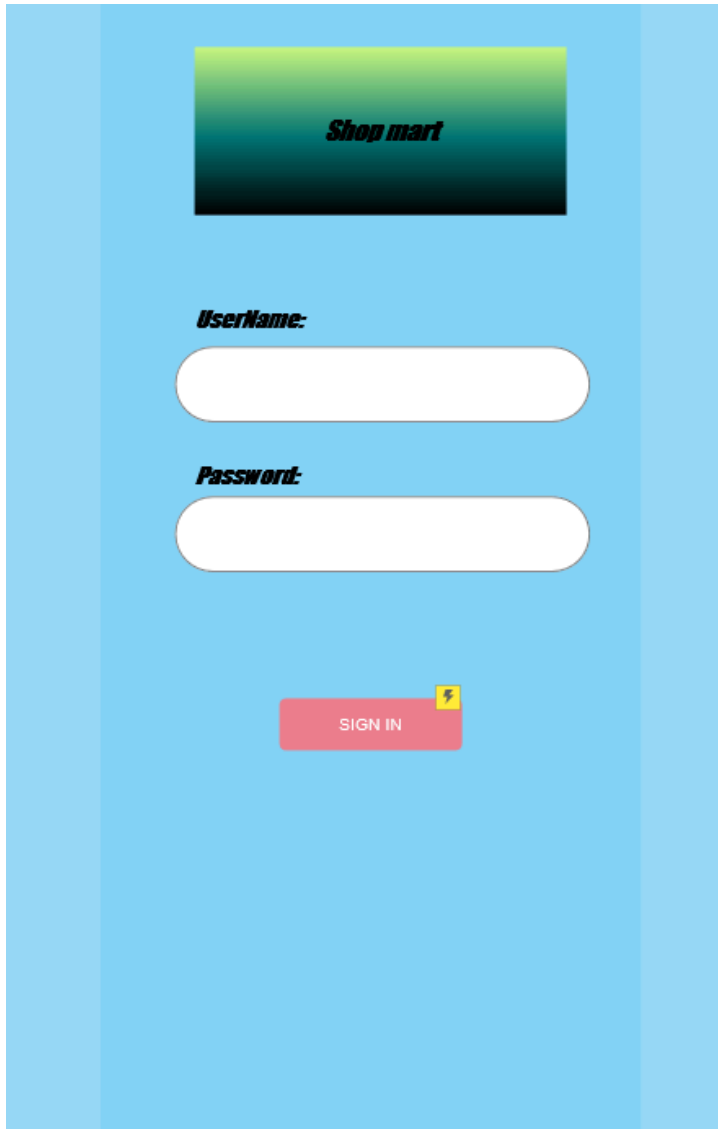
1. Finalize and Export:

- Finalize the design and interactions.
- Export the prototype as an HTML file or share it via Axure Cloud.

2. User Training and Support:

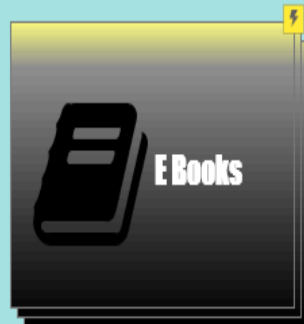
- Conduct training sessions to familiarize users with the new interface.
- Provide documentation and support for any issues.

OUTPUT:



The image shows a login interface for 'Shop mart'. At the top, there is a logo with the text 'Shop mart' in a bold, italicized font, set against a green-to-black gradient background. Below the logo, the text 'UserName:' is followed by a white, rounded rectangular input field. Underneath that, the text 'Password:' is followed by another white, rounded rectangular input field. At the bottom of the form, there is a red rectangular button with the text 'SIGN IN' in white capital letters. A small yellow lightning bolt icon is positioned to the right of the button. The entire interface is centered on a light blue background.

Shop Mart



Order Confirmation

Product Name:

order now



RESULT:

The lifecycle simulation using the RAD model in axure RP is implemented and executed.

**Simulate the life cycle stages for UI design using the RAD
model and develop a small interactive interface using
OpenProj**

AIM:

The aim is to recreate the lifecycle stages of UI design using the RAD model and design a small interactive interface with OpenProj

PROCEDURE:

Tool Link: <https://sourceforge.net/projects/openproj/>

Step 1: Requirements Planning

1. Gather Requirements:

- Identify key features and functionalities needed for your interface.
- Example: A simple "Login" and "Register" interface with debug logs.

2. Define Use Cases:

- Specify use cases for user login and registration.
- Example: User logs in with valid credentials, user registers with a new account.

Output in OpenProj:

- Create a new project.
- Add tasks: "Gather Requirements" and "Define Use Cases."
- Set durations and dependencies for each task.

Step 2: User Design

1. Sketch Initial Designs:

- Draw rough sketches of the "Login" and "Register" screens on paper.

2. Create Digital Wireframes:

- Use a tool like Figma or Sketch to create digital wireframes.

Example Wireframes:

1. **Login Screen:** Username field, Password field, Login button, Register link.
2. **Register Screen:** Username field, Email field, Password field, Confirm Password field, Register button.

Output in OpenProj:

- Add tasks: "Sketch Initial Designs" and "Create Digital Wireframes."
- Allocate time and resources to complete these tasks.

Step 3: Rapid Prototyping

1. Develop Prototypes:

- Use a tool like Axure RP to convert wireframes into interactive prototypes.

2. Test Prototypes:

- Share prototypes with stakeholders for feedback.
- Collect feedback and iterate on the design.

Output:

- Interactive prototypes for "Login" and "Register" screens.

Output in OpenProj:

- Add tasks: "Develop Prototypes" and "Test Prototypes."
- Set dependencies and milestones.

Step 4: User Acceptance/Testing

1. Review Prototype:

- Conduct user and stakeholder reviews.

2. Conduct Usability Testing:

- Perform usability testing and document feedback.

Output:

- Documented feedback and test results.

Output in OpenProj:

- Add tasks: "Review Prototype" and "Usability Testing."
- Track progress and resources.

Step 5: Implementation

1. Develop Functional Interface:

- Implement final designs and functionalities based on feedback.

2. Integrate Backend (if required):

- Connect the UI with backend services for tasks like user authentication.

OUTPUT:

The image displays two side-by-side wireframe boxes representing a web application's user interface. The left box is titled 'REGISTER' and contains four input fields stacked vertically: the first contains the text '230701292', the second is labeled 'Email', the third is labeled 'Password', and the fourth is labeled 'Confirm Password'. Below these fields is a 'Register' button. The right box is titled 'LOGIN' and contains two input fields stacked vertically: the first contains the text '230701292' and the second is labeled 'Password'. Below these fields is a 'Login' button.

REGISTER	LOGIN
230701292	230701292
Email	Password
Password	
Confirm Password	
Register	Login

RESULT:

Hence the life cycle stages for UI design using the RAD model and develop a small interactive interface using OpenProj is implemented and executed.

Exercise 6

Date:

Experiment with different layouts and color schemes for an app.

Collect user feedback on aesthetics and usability using GIMP(GNU Image Manipulation Program (GIMP))

AIM:

The aim is to trial different app layouts and color schemes and evaluate user feedback on aesthetics and usability using GIMP.

PROCEDURE:

Tool Link: <https://www.gimp.org/>

Step 1: Install GIMP

- **Download and Install:** Download GIMP from GIMP Downloads and install it on your computer.

Step 2: Create a New Project

1. **Open GIMP:**
 - Launch the GIMP application.
2. **Create a New Canvas:**
 - Go to File -> New to create a new project.
 - Set the dimensions for your app layout (e.g., 1080x1920 pixels for a standard mobile screen).

Step 3: Design the Base Layout

1. **Create the Base Layout:**

- Use the Rectangle Select Tool to create sections for different parts of your app (e.g., header, content area, footer).
- Fill these sections with basic colors using the Bucket Fill Tool.

Example Output: A base layout with defined sections for header, content, and footer.

2. Add UI Elements:

- **Text Elements:** Use the Text Tool to add text elements like headers, buttons, and labels.
- **Interactive Elements:** Use the Brush Tool or Shape Tools to draw buttons, input fields, and other interactive elements.

Example Output: A layout with labeled sections and basic UI elements.

3. Organize Layers:

- Use layers to separate different UI elements. This allows you to easily modify or experiment with individual components.
- Name each layer according to its content (e.g., Header, Button1, InputField).

Step 4: Experiment with Color Schemes

1. Create Color Variants:

- **Duplicate Layout:** Duplicate the base layout by right-clicking on the image tab and selecting Duplicate.
- **Change Colors:** Use the Bucket Fill Tool or Colorize Tool to change the colors of the UI elements in each duplicate.

Example Output: Multiple color variants of the same layout.

2. Save Each Variant:

- Save each color variant as a separate file (e.g., Layout1.png, Layout2.png, etc.).
- Go to File -> Export As and choose the file format (e.g., PNG).

Step 5: Collect User Feedback

1. Prepare a Feedback Form:

- **Create Form:** Create a feedback form using tools like Google Forms or Microsoft Forms.
- **Include Questions:** Include questions about the aesthetics and usability of each layout and color scheme.

2. Share the Variants:

- **Distribute Files:** Share the image files of the different layouts and color schemes with your users.
- **Provide Instructions:** Provide clear instructions on how to view each variant and how to fill out the feedback form.

3. Gather Feedback:

- Collect responses from users regarding their preferences and suggestions.
- Analyze the feedback to determine which layout and color scheme are most preferred.

Step 6: Iterate and Refine

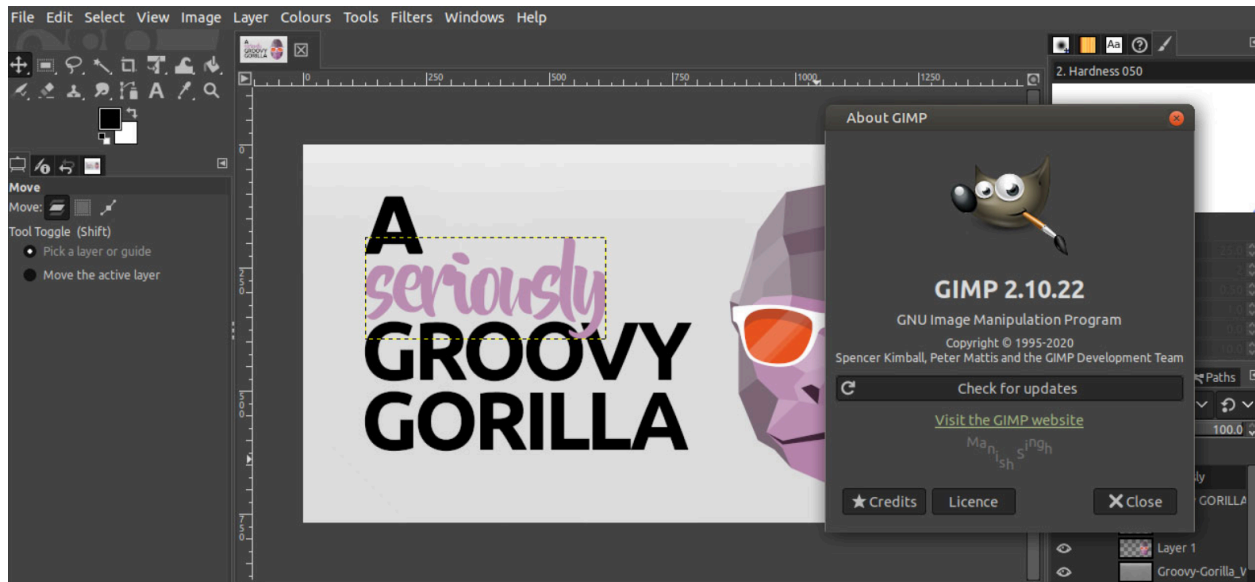
1. Refine the Design:

- Based on the feedback, make necessary adjustments to the layout and color scheme.
- Experiment with additional variations if needed.

2. Final Testing:

- Conduct a final round of testing with the refined design to ensure usability and aesthetic satisfaction.

OUTPUT:



RESULT:

Hence the different layout and colour schemes using GIMP is implemented and executed.

Develop low-fidelity paper prototypes for a banking app and convert them into digital wireframes using Pencil Project

AIM:

The aim is to develop low-fidelity paper prototypes for a banking app and convert them into digital wireframes with Pencil Project.

PROCEDURE:

Tool Link: <https://pencil.evolus.vn/>

Step 1: Create Low-Fidelity Paper Prototypes

1. Define the Purpose and Features:

- Identify the core features of the banking app (e.g., login, account balance, transfers, bill payments).

2. Sketch Basic Layouts:

- Use plain paper and pencils to sketch basic screens.
- Focus on primary elements like buttons, menus, and forms.

3. Iterate and Refine:

- Get feedback from users or stakeholders.
- Iterate on your sketches to improve clarity and functionality.

Step 2: Convert Paper Prototypes to Digital Wireframes Using Pencil Project

1. Install Pencil Project:

- Download and install Pencil Project from the official website.

2. Create a New Document:

- Open Pencil Project and create a new document.

3. **Add Screens:**

- Click on the "Add Page" button to create different screens (e.g., Login, Dashboard, Transfer).

4. **Use Stencils and Shapes:**

- Use the built-in stencils and shapes to create UI elements.
- Drag and drop elements like buttons, text fields, and icons onto your canvas.

5. **Organize and Align:**

- Arrange and align the elements to match your paper prototype.
- Ensure that the design is user-friendly and intuitive.

6. **Link Screens:**

- Use connectors to link different screens together.
- Create navigation flows to show how users will interact with the app.

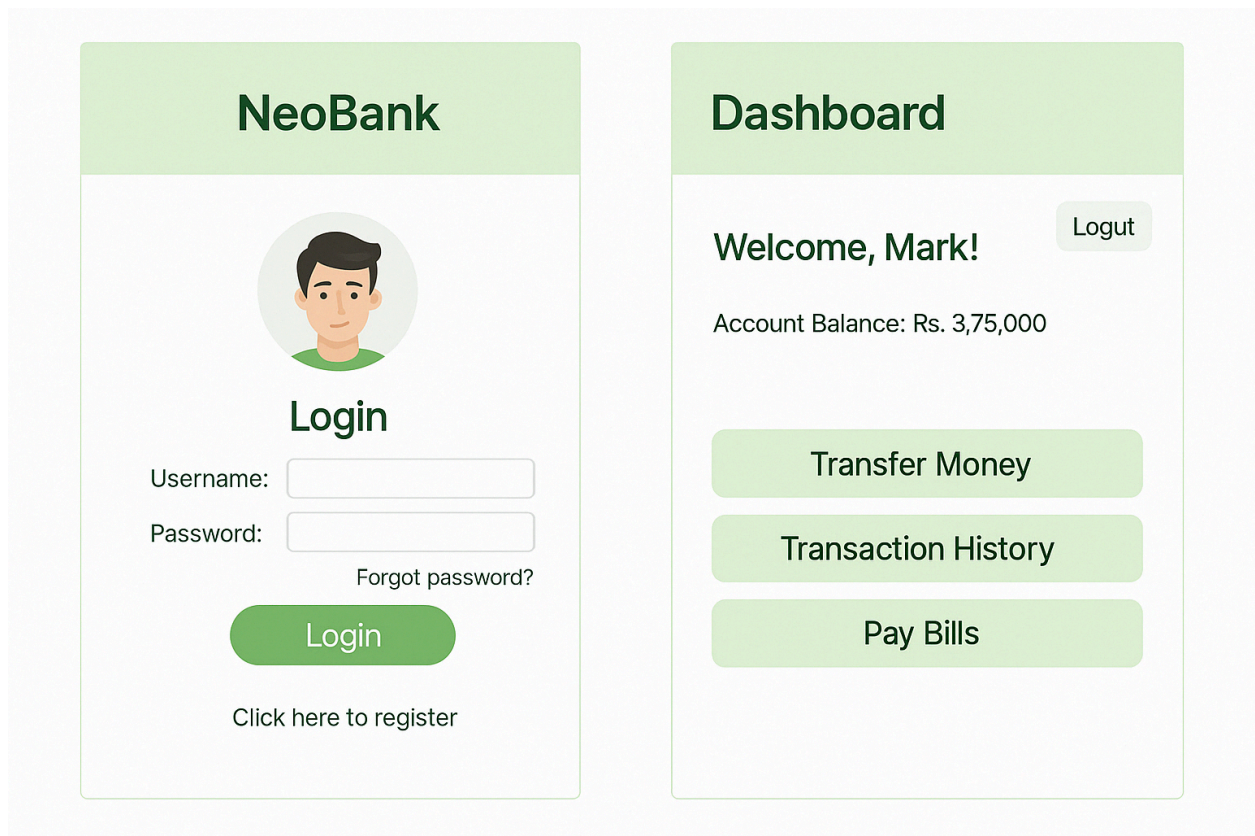
7. **Add Annotations:**

- Include annotations to explain the functionality of different elements.

8. **Export Your Wireframes:**

- Once satisfied with your digital wireframes, export them in your preferred format (e.g., PNG, PDF).

OUTPUT:



RESULT:

Hence low-fidelity paper prototypes for a banking app and convert them into digital wireframes using Pencil Project is implemented and verified

Exercise 7B

Date:

Develop low-fidelity paper prototypes for a banking app and convert them into digital wireframes using Inkscape

AIM:

The aim is to construct low-fidelity paper prototypes for a banking app and digitize them into wireframes using Inkscape.

PROCEDURE:

Tool Link: <https://inkscape.org/>

Step 1: Create Low-Fidelity Paper Prototypes

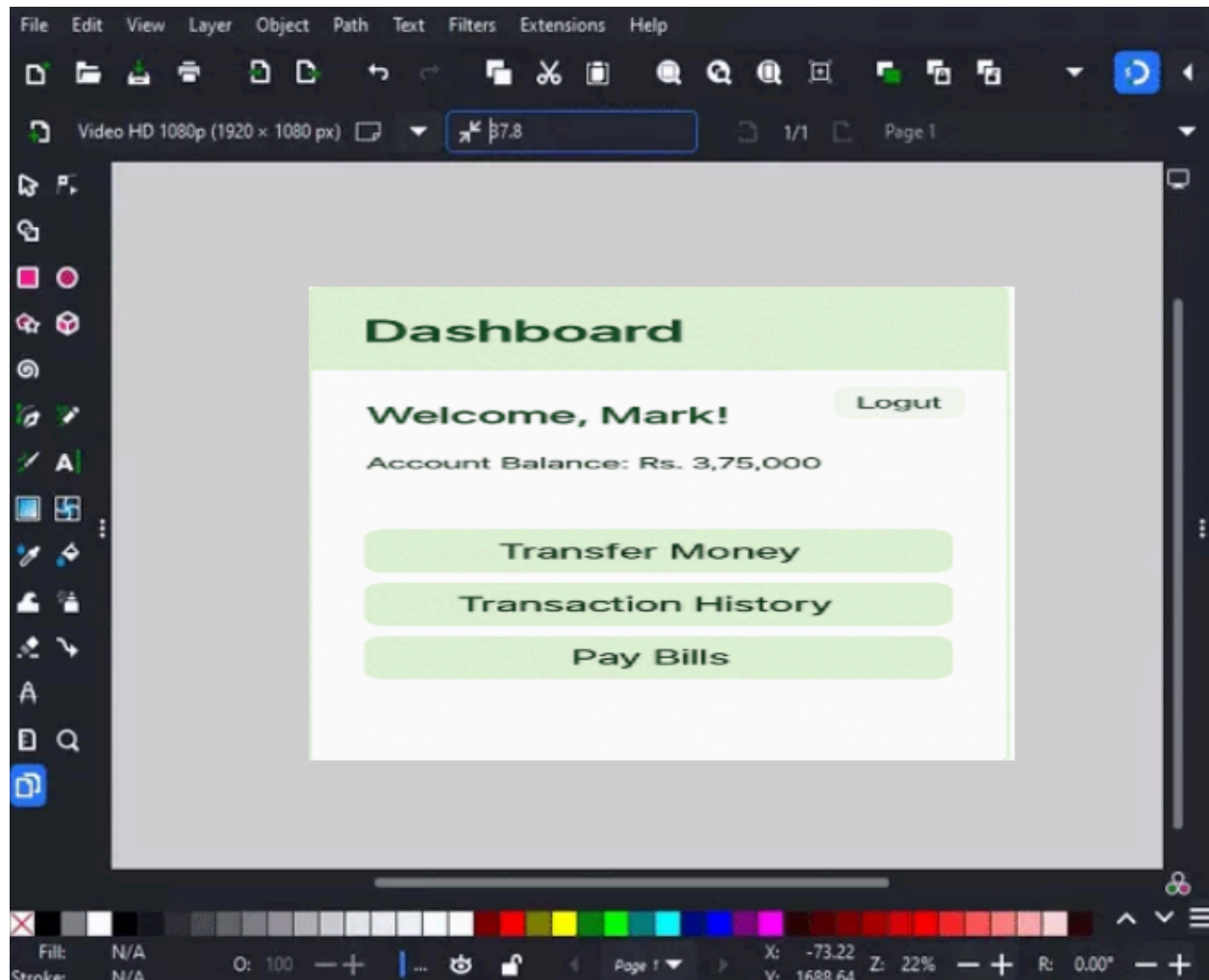
1. Identify Core Features:
 - Determine the essential features of the banking app (e.g., login, dashboard, account management, transfers).
2. Sketch Basic Layouts:
 - Use plain paper and pencils to sketch the main screens.
 - Focus on the primary elements like buttons, navigation menus, and input fields.
3. Iterate and Refine:
 - Get feedback from users or stakeholders.
 - Make necessary adjustments to improve clarity and functionality.

Step 2: Convert Paper Prototypes to Digital Wireframes Using Inkscape

1. Install Inkscape:
 - Download and install Inkscape from the official website.
2. Create a New Document:

- Open Inkscape and create a new document by clicking on File > New.
3. Set Up the Document:
 - Set the dimensions and grid for your design. Go to File > Document Properties to adjust the size.
 - Enable the grid by going to View > Page Grid.
 4. Draw Basic Shapes:
 - Use the rectangle and ellipse tools to draw the basic shapes for your UI elements (e.g., buttons, input fields, icons).
 5. Add Text:
 - Use the text tool to add labels and placeholder text to your elements.
 6. Organize and Align:
 - Arrange and align the elements to match your paper prototype.
 - Use the alignment and distribution tools to keep everything organized.
 7. Group Elements:
 - Select related elements and group them together using Object > Group.
 - This helps keep your design organized and easy to edit.
 8. Create Multiple Screens:
 - Duplicate your base layout to create different screens (e.g., login, dashboard, transfer).
 - Use Edit > Duplicate to create copies of your elements and arrange them for each screen.
 9. Link Screens (Optional):
 - If you want to show navigation flows, you can add arrows or other indicators to demonstrate how users will move between screens.
 10. Export Your Wireframes:
 - Once you're satisfied with your digital wireframes, export them by going to File > Export PNG Image.
 - Choose the appropriate settings and export each screen as needed.

OUTPUT:



RESULT:

Hence the low-fidelity paper prototypes for a banking app and convert them into digital wireframes using Inkscape is implemented and executed.

Excercise 8a

Date:

Create storyboards to represent the user flow for a mobile app **(e.g., food delivery app) using Balsamiq**

AIM:

The aim is to create storyboards representing the user flow for a mobile app, such as a food delivery app, using Balsamiq.

PROCEDURE:

Tool Link: <https://balsamiq.com/>

Step 1: Define the User Flow

1. Identify Key Screens:

- List the main screens your app will have (e.g., Home, Menu, Cart, Checkout, Order Confirmation).

2. Map the User Journey:

- Understand the typical user journey through these screens (e.g., browsing menu, adding items to cart, checking out).

Step 2: Create Storyboards Using Balsamiq

1. Install Balsamiq:

- Download and install Balsamiq from the <https://balsamiq.com/> website.

2. Create a New Project:

- Open Balsamiq and create a new project.

3. Add Wireframe Screens:

- Use the “+” button to add new wireframe screens for each key screen in your app.

4. **Design Each Screen:**

- Use Balsamiq's components to design the UI for each screen.
- Include basic elements like buttons, text fields, and images.

5. **Organize the Flow:**

- Arrange the screens in the order users will navigate through them.
- Connect the screens with arrows to represent user actions.

Example Screens for Food Delivery App

1. **Home Screen:**

- Search bar for finding restaurants
- Categories for different cuisines

2. **Menu Screen:**

- List of food items with images, names, and prices
- Add to Cart buttons

3. **Cart Screen:**

- Items added to the cart with quantity and total price
- Checkout button

4. **Checkout Screen:**

- Delivery address form
- Payment options
- Place Order button

5. **Order Confirmation Screen:**

- Order summary
- Estimated delivery time

Example Output

Here's how the wireframes might look:

Home Screen

- **Search Bar:** Allows users to search for restaurants.
- **Categories:** Buttons for different cuisines (e.g., Italian, Chinese).

Menu Screen

- **Food Items List:** Displays food items with images, names, and prices.
- **Add to Cart:** Button to add items to the cart.

Cart Screen

- **Items Added:** Lists items added to the cart with quantity and prices.
- **Checkout Button:** Proceed to checkout.

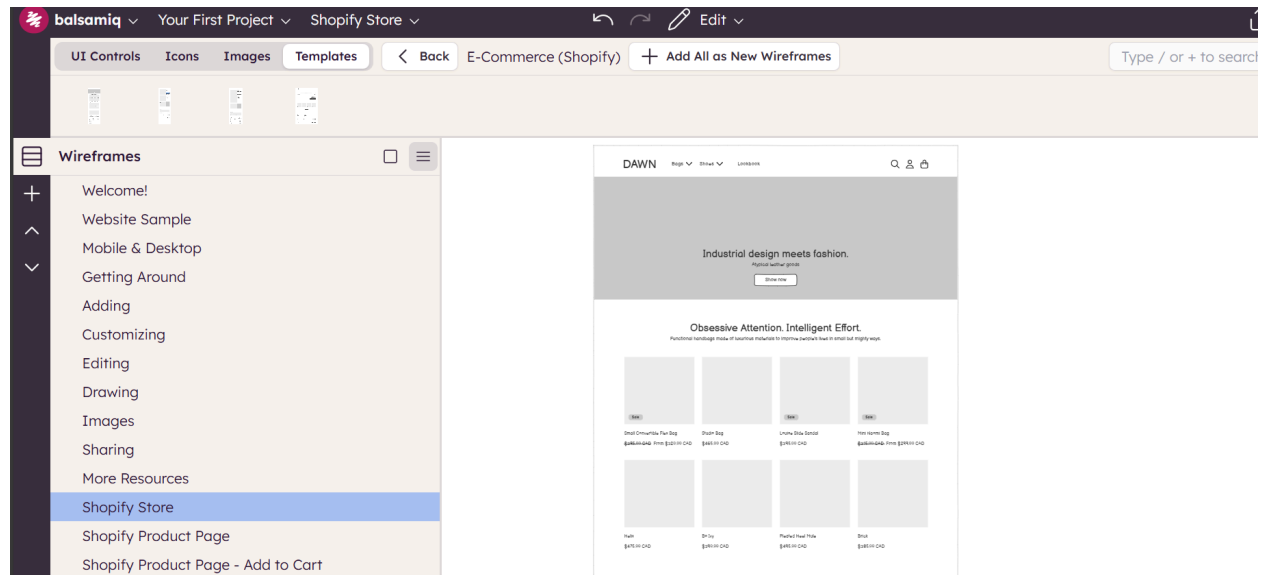
Checkout Screen

- **Delivery Address Form:** Users enter their delivery address.
- **Payment Options:** Choose between different payment methods.
- **Place Order Button:** Finalize the order.

Order Confirmation Screen

- **Order Summary:** Shows the order details.
- **Estimated Delivery Time:** Provides an estimated delivery time.

OUTPUT:



✓ Item added to your cart

Mini Naomi Bag
Color: Pastel

View Cart

Checkout

[Continue shopping](#)

- 1 +

Add to cart

Buy it now

Mini Naomi Bag is perfectly proportioned for your essentials like a big-screen phone, cardholder, and lipstick. You can wear it around your waist or slung over one shoulder, and it comes with an extra shoulder strap, so you can also wear it crossbody.

 Item Name Shipping and Returns Dimensions Care Instructions Share

A new kind of bag.

Unexpected shapes with smart details, functionality, a new luxury feel with a contemporary price point.

Your cart

[Continue shopping](#)

PRODUCT	QUANTITY	TOTAL
 Mini Bag Color: Pastel	1 + -	\$215.00

Subtotal 300.00 R\$
Taxes and shipping calculated at checkout

[Checkout](#)

You may also like

[View all](#)

Quick Links

[Bags](#)
[Shirts](#)
[Luggage](#)

Info

[About us](#)
[Contact us](#)
[Shipping policy](#)
[Blog](#)

Our mission

Quality materials, good designs,
craftsmanship and sustainability.

Subscribe to our emails



Country / Region



RESULT:

Hence the storyboard for mobile application using balsamiq is implemented and executed.

Exercise 8B

Date:

Create storyboards to represent the user flow for a mobile app **(e.g., food delivery app) using OpenBoard**

AIM:

To map out the user flow for a mobile app (e.g., a food delivery app), storyboards will be designed using OpenBoard.

PROCEDURE:

Tool Link: <https://openboard.ch/download.en.html>

Step 1: Define the User Flow

1. Identify Key Screens:

- List the main screens your app will have (e.g., Home, Menu, Cart, Checkout, Order Confirmation).

2. Map the User Journey:

- Understand the typical user journey through these screens (e.g., browsing menu, adding items to cart, checking out).

Step 2: Create Storyboards Using OpenBoard

1. Install OpenBoard:

- Download and install OpenBoard from the official website.

2. Create a New Document:

- Open OpenBoard and create a new document.

3. Add Frames for Each Screen:

- Use the drawing tools to create frames representing each key screen of your app.

4. Sketch Each Screen:

- Use the pen or shape tools to draw basic elements for each screen.

- Focus on major UI components like buttons, text fields, and icons.

5. Organize the Flow:

- Arrange the frames in a sequence that represents the user journey.
- Use arrows or lines to show navigation paths between screens.

Example Screens for Food Delivery App

1. Home Screen:

- Search bar for finding restaurants
- Categories for different cuisines

2. Menu Screen:

- List of food items with images, names, and prices
- Add to Cart buttons

3. Cart Screen:

- Items added to the cart with quantity and total price
- Checkout button

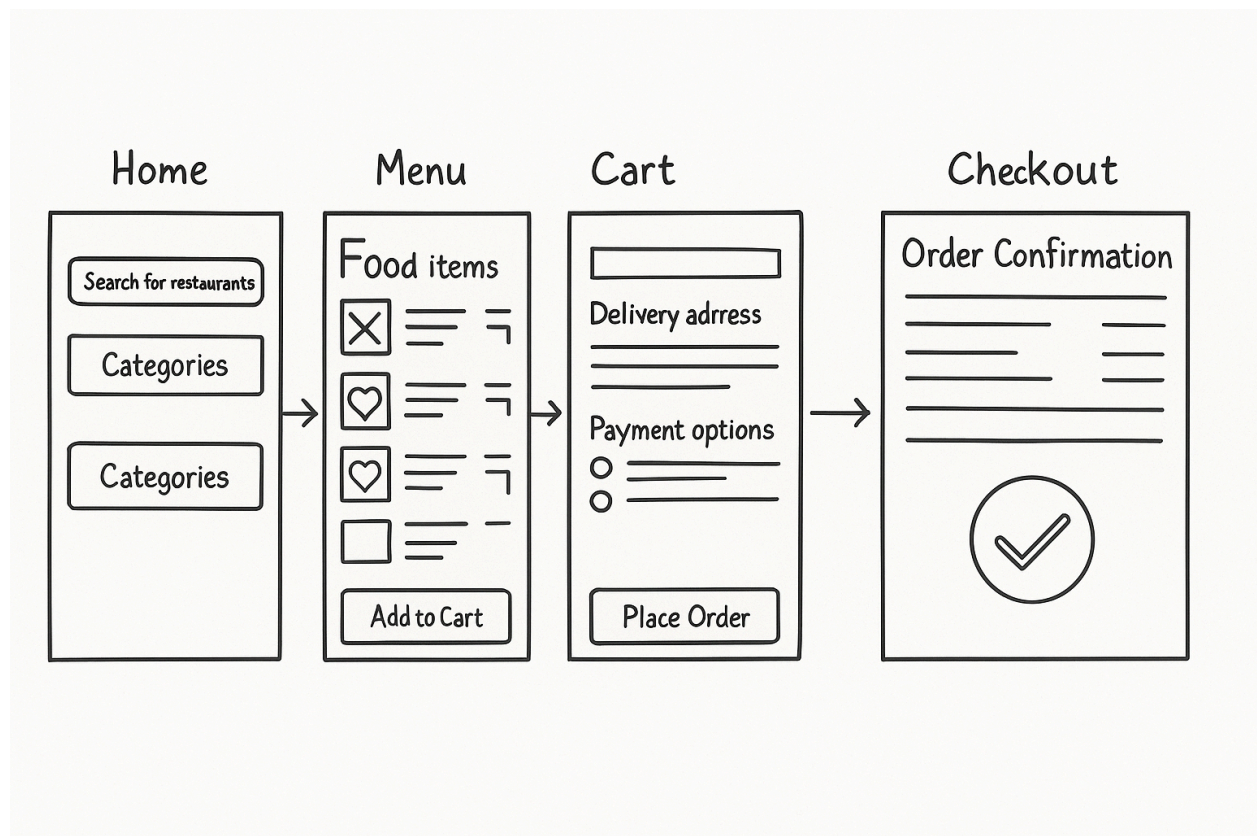
4. Checkout Screen:

- Delivery address form
- Payment options
- Place Order button

5. Order Confirmation Screen:

- Order summary
- Estimated delivery time

OUTPUT:



RESULT:

Hence the Storyboards using Open Board are implemented and executed.

Exercise 9

Date:

Design input forms that validate data (e.g., email, phone number) and display error messages using HTML/CSS, JavaScript (with Validator.js)

AIM:

The aim is to design input forms that validate data, such as email and phone number, and display error messages using HTML/CSS and JavaScript with Validator.js.

PROCEDURE:

Step 1: Setting Up the HTML Form

Start by creating an HTML form with input fields for the email and phone number.

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form Validation</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <form id="myForm">
      <label for="email">Email:</label>
```

```
<input type="email" id="email" name="email" required>
<span id="emailError" class="error"></span>

<label for="phone">Phone Number:</label>
<input type="text" id="phone" name="phone" required>
<span id="phoneError" class="error"></span>

<button type="submit">Submit</button>
</form>
</div>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.6.0/validator.min.js"></script>
<script src="script.js"></script>
</body>
</html>
```

Step 2: Styling the Form with CSS

Next, add some basic styling to make the form look nice.

css

```
/* style.css */
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
```

```
}
```

```
.container {  
  background-color: white;  
  padding: 20px;  
  border-radius: 5px;  
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);  
}
```

```
form {  
  display: flex;  
  flex-direction: column;  
}
```

```
label {  
  margin-bottom: 5px;  
}
```

```
input {  
  margin-bottom: 10px;  
  padding: 10px;  
  border: 1px solid #ccc;  
  border-radius: 3px;  
}
```

```
button {  
  padding: 10px;  
  background-color: #28a745;  
  color: white;
```

```
border: none;
border-radius: 3px;
cursor: pointer;
}

button:hover {
  background-color: #218838;
}

.error {
  color: red;
  font-size: 0.875em;
}
```

Step 3: Adding JavaScript for Validation

Finally, add JavaScript to validate the input fields using Validator.js and display error messages.

javascript

```
/* script.js */

document.getElementById('myForm').addEventListener('submit', function (e) {
  e.preventDefault();

  let email = document.getElementById('email').value;
  let phone = document.getElementById('phone').value;

  let emailError = document.getElementById('emailError');
  let phoneError = document.getElementById('phoneError');
```

```
// Clear previous error messages
emailError.textContent = "";
phoneError.textContent = "";

// Validate email
if (!validator.isEmail(email)) {
    emailError.textContent = 'Please enter a valid email address.';
}

// Validate phone number
if (!validator.isMobilePhone(phone, 'any')) {
    phoneError.textContent = 'Please enter a valid phone number.';
}

// If no errors, submit the form (for demonstration purposes, we'll just log the values)
if (validator.isEmail(email) && validator.isMobilePhone(phone, 'any')) {
    console.log('Email:', email);
    console.log('Phone:', phone);
}
});
```

OUTPUT:

Contact Form

Email

Phone Number

RESULT:

Hence the input forms to validate data are implemented and executed using html,css and js.

Exercise 10

Date:

Create a data visualization (e.g., pie charts, bar graphs) for an inventory management system using javascript

AIM:

The aim is to create data visualizations, such as pie charts and bar graphs, for an inventory management system using JavaScript.

PROCEDURE:

Step 1: Set Up Your HTML File

First, create an HTML file to hold your canvas for the chart and include Chart.js.

html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Inventory Management Visualization</title>
```

```
  <style>
```

```
    body {
```

```
      font-family: Arial, sans-serif;
```

```
      text-align: center;
```

```
      margin: 50px;
```

```
    }
```

```
    canvas {
```

```
      margin: 20px auto;
```

```

    }
  </style>
</head>
<body>
  <h1>Inventory Management System</h1>
  <canvas id="pieChart" width="400" height="400"></canvas>
  <canvas id="barChart" width="400" height="400"></canvas>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <script src="script.js"></script>
</body>
</html>

```

Step 2: Create the JavaScript File for Charts

Next, create a JavaScript file (script.js) to handle the data visualization logic.

```

javascript
// script.js

// Data for the inventory
const inventoryData = {
  labels: ['Electronics', 'Clothing', 'Home Appliances', 'Books', 'Toys'],
  datasets: [
    {
      label: 'Items in Stock',
      data: [200, 150, 100, 80, 50],
      backgroundColor: [
        '#FF6384',
        '#36A2EB',
        '#FFCE56',

```

```
        '#4BC0C0',  
        '#9966FF'  
    ],  
    }  
]  
};
```

// Creating the Pie Chart

```
const ctxPie = document.getElementById('pieChart').getContext('2d');  
const pieChart = new Chart(ctxPie, {  
    type: 'pie',  
    data: inventoryData,  
    options: {  
        responsive: true,  
        title: {  
            display: true,  
            text: 'Inventory Distribution'  
        }  
    }  
});
```

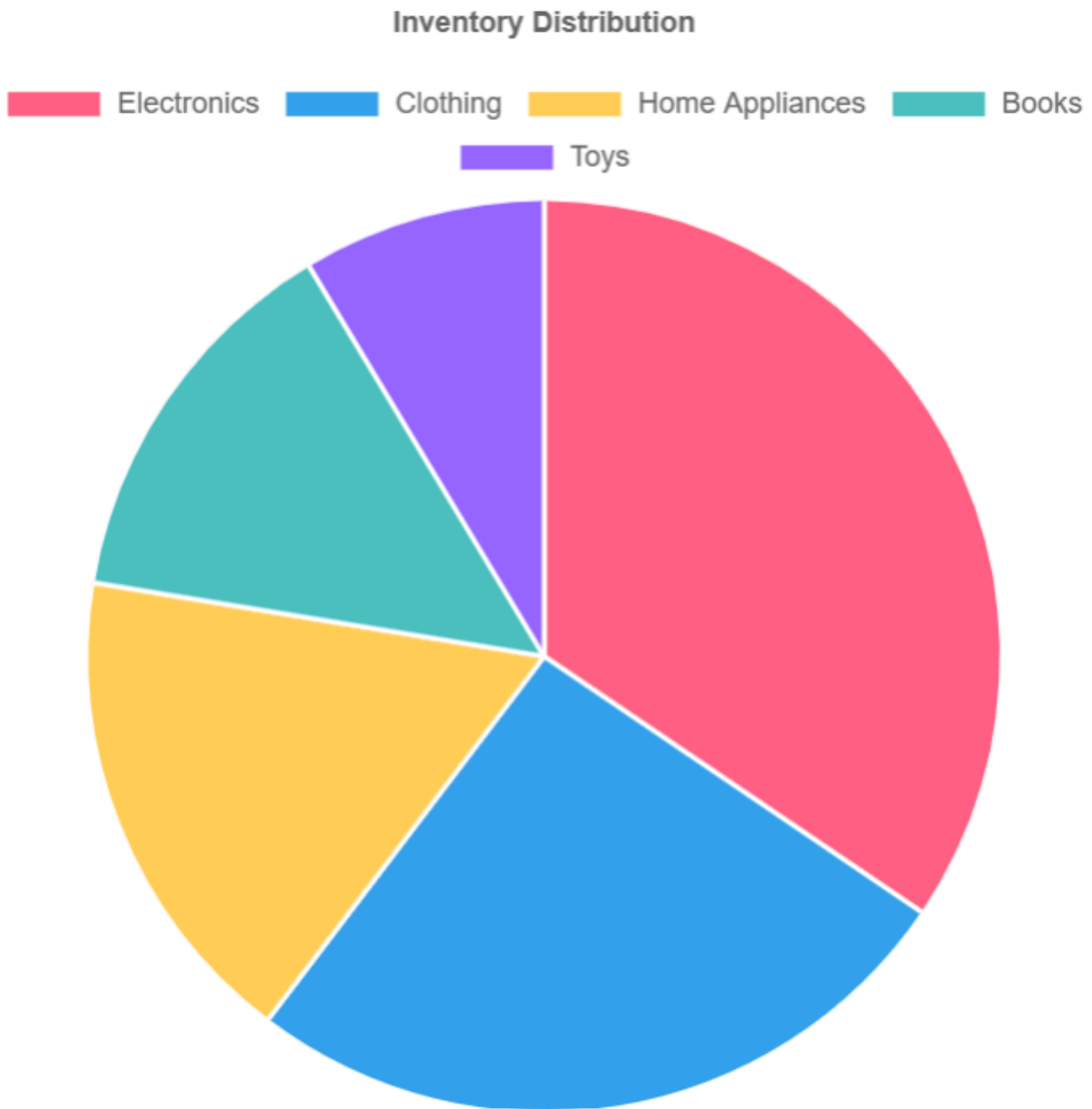
// Creating the Bar Chart

```
const ctxBar = document.getElementById('barChart').getContext('2d');  
const barChart = new Chart(ctxBar, {  
    type: 'bar',  
    data: inventoryData,  
    options: {  
        responsive: true,  
        title: {
```

```
    display: true,  
    text: 'Items in Stock by Category'  
  },  
  scales: {  
    yAxes: [{  
      ticks: {  
        beginAtZero: true  
      }  
    }]  
  }  
}  
});
```

OUTPUT:

Inventory Management System



RESULT:

Hence the Inventory management system with pie charts is implemented and executed.